

Amlogic NPU Quick Start Guide

Version: v0.1

Release Date: 2026-06-25

Copyright

© 2026 Amlogic. All rights reserved. No part of this document may be reproduced, transmitted, transcribed, or translated into any language in any form or by any means without the written permission of Amlogic.

Trademarks

 **Amlogic**, and other Amlogic icons are trademarks of Amlogic companies. All other trademarks and registered trademarks are property of their respective holders.

Disclaimer

Amlogic may make improvements and/or changes in this document or in the product described in this document at any time and without notice.

This product is not intended for use in medical, life saving, or life sustaining applications, nor in applications where failure or malfunction of an Amlogic product can reasonably be expected to result in personal injury, death or severe property or environmental damage.

Circuit diagrams and other information relating to products of Amlogic are included as a means of illustrating typical applications. Consequently, complete information sufficient for production design is not necessarily given. Though every effort has been made to ensure that this document is current and accurate, Amlogic makes no representations or warranties with respect to the accuracy or completeness of the contents presented in this document.

Amlogic products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Amlogic accepts no liability for any vulnerability.

Use of the materials may require a license of intellectual property from a third party or from Amlogic. This document conveys no express or implied licenses to any intellectual property rights belonging to Amlogic or any other party. Any links to third-party websites included in this document are for your convenience only. Amlogic does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Amlogic shall in no event be liable for any claims relating to or arising out of this document or any use or inability to use hereof.

Contact Information

- Website: www.amlogic.com
- Pre-sales consultation: contact@amlogic.com
- Technical support: support@amlogic.com

Revision History

Version	Description
v0.1 (2026-06-25)	This is the First Version.

Contents

1 Document Description

- 1.1 Document Purpose
- 1.2 Applicable Platforms
- 1.3 Keyword Description

2 Development Environment Verification

- 2.1 Version Verification
- 2.2 System Environment and Architecture Verification
- 2.3 Quick Verification
 - 2.3.1 General Case Verification
 - 2.3.2 LLM Quick Verification

3 Toolkit Installation

- 3.1 Deployment Methods and Toolkit Introduction
- 3.2 Download AMLNN Related Repositories
 - 3.2.1 amlnn-toolkit Introduction
 - 3.2.2 amlnn-model-playground Introduction
- 3.3 Install Toolkit Environment
 - 3.3.1 Ubuntu PC-side amlnn-toolkit Installation
 - 3.3.2 Board-side Armbian System Installation of amlnn-edge-toolkit

4 Board-side Conversion and Inference

- 4.1 Preparation
- 4.2 Model Conversion and Inference
- 4.3 USB Camera Real-time Acquisition Demo Acquisition Guide

5 PC-side Model Conversion

- 5.1 Preparation
 - 5.1.1 Download Model
 - 5.1.2 Platform Type Confirmation
- 5.2 Model Conversion

6 ADB Board-connected Inference

- 6.1 Install ADB Tool
- 6.2 Start nnserver
- 6.3 Run AMLNN Python Demo

7 Board-side C/C++ Inference

- 7.1 Install Compilation Toolchain
 - 7.1.1 Install CMake
 - 7.1.2 Install Cross-compilation Toolchain
 - 7.1.2.1 Android System: Install and Configure NDK
 - 7.1.2.2 Buildroot System: Install GCC Cross-compilation Toolchain
 - 7.1.2.3 Armbian/Yocto System: Install Cross-compilation Toolchain
- 7.2 Run AMLNN C/C++ Demo
 - 7.2.1 Android System C/C++ Demo Compilation and Execution
 - 7.2.2 Buildroot System C/C++ Demo Compilation and Execution
 - 7.2.3 Armbian/Yocto System C/C++ Demo Compilation and Execution
 - 7.2.4 Expected Results

8 FAQ

- 8.1 adb devices Cannot Recognize Device
- 8.2 nnserver Service Related Issues

1 Document Description

1.1 Document Purpose

This document aims to help developers quickly get started with deep learning model deployment on the Amlogic NPU (ADLA (amlogic Deep Learning Accelerator)) platform. Whether you are new to embedded AI development or looking to migrate existing models to the Amlogic platform, this guide provides a complete workflow from environment setup, model conversion, compilation and deployment, to runtime verification.

The examples used in this document have been integrated into the AMLNN Model Playground.

- [AMLNN Toolkit](#): The core hub for Amlogic NPU development, covering the complete software stack required for deploying AI models on Amlogic platforms—from high-level Python runtime to low-level C/C++ SDK and reference examples.
- [AMLNN Model Playground](#): A model library built on top of the AMLNN Toolkit, providing a rich set of mainstream algorithm examples, including complete model conversion scripts and full code for inference using Python API and C/C++ API.

1.2 Applicable Platforms

This guide applies to the following Amlogic NPU platforms:

- C308L / C302X
- S928X
- A311D2
- T968D4
- S905X5 / S905D5
- C302X2
- C305X2/A123X
- A311Y3

Supported operating systems for each platform:

	Buildroot	Android	Armbian(Debian)	Yocto
C308L/C302X	✓	×	×	×
S928X	×	✓	×	✓
A311D2	×	✓	×	✓
T968D4	×	✓	×	✓
S905X5/S905D5	×	✓	×	✓
C302X2	✓	×	×	×
A311Y3	×	✓	✓	✓
C305X2/A123X	✓	×	×	×

1.3 Keyword Description

- **ADLA:** ADLA (amlogic Deep Learning Accelerator), the model file running on the Amlogic platform, with the suffix .adla.
- **Armbian (Debian):** Referred to as Armbian in subsequent sections of this document.

2 Development Environment Verification

2.1 Version Verification

Confirm whether the ADLA NPU driver version and AML Runtime library version meet the requirements. After connecting to the development board via adb, perform the following operations to confirm the version status.

Step 1: NPU driver version verification

- **Execute command:** adb shell cat /sys/class/adla/adla0/device/kmd_version
- **Reference result is as follows:**

```
a9_a311y3:/ # cat /sys/class/adla/adla0/device/kmd_version
ADLA Version: 2.0.2.0.0
a9_a311y3:/ # |
```

- **Expected result:** Version >= 2.0.2

Step 2: NNSDK version verification

- **Execute command**

```
adb shell

#for Android
strings /vendor/lib64/libnnsdk.so | grep NNSDK

#for Linux(buildroot/armbian/yocto)
strings /usr/lib/libnnsdk.so | grep NNSDK
```

- **Reference result is as follows:**

```
a9_a311y3:/ # strings /vendor/lib64/libnnsdk.so | grep NNSDK
NNSDK Version: v3.0.0, 2026.05
NNSDK2 Version: v1.0.0, 2026.05
vendor.NNSDK2_LOG_LEVEL
NNSDK2
a9_a311y3:/ # |
```

- **Expected result:**
 - NNSDK Version >= 3.0.0
 - NNSDK2 Version >= 1.0.0

2.2 System Environment and Architecture Verification

Confirm the system environment and architecture of the board side to provide guidance for selecting the corresponding quick-start test files, nnserver, and cross-compilation toolchains in subsequent sections.

Step 1: First confirm whether it is an Android platform or a Linux platform

Execute command: `adb shell getprop ro.product.cpu.abi`

Judgment logic:

- **Output** `arm64-v8a`: confirmed as 64-bit Android system
- **Output** `armeabi-v7a`: confirmed as 32-bit Android system
- **Error** `command not found`: judged as Linux system, please continue to Step 2

Step 2: Further confirmation of Linux system (Buildroot / Yocto / Armbian) environment

Step 2.1: Confirm whether it is an Armbian / Yocto system

Execute command: `cat /version.txt`

Reference output example (taking Armbian as an example)

```
image name: a9-by401-bookworm-20260707030949
BRANCH=
VERSION=Linux/Debian/Armbian-PB#786
SPIN=Linux/Debian/Armbian-PB#786
BUILD_TIME="2026-07-07 03:27:41"
Generated on Tue Jul 7 03:27:41 UTC 2026

Compiler:
aarch64-none-linux-gnu-gcc (Arm GNU Toolchain 11.3.Rel1) 11.3.1 20220712
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Judgment logic:

- If this file exists: judged as an Armbian/Yocto system, and then determine whether it is a 32-bit/64-bit system based on the Compiler version information. The reference example corresponds to an Armbian 64-bit (AArch64) system.
- If the file does not exist: command execution fails, judged as a Buildroot system, please continue to Step 2.2

Note: The above confirmation method is only applicable to Amlogic official release boards. Third-party development boards should confirm whether the same version files and judgment logic are supported.

Step 2.2: Confirm Buildroot system architecture

Execute command: `adb shell uname -m`

Judgment logic:

- `aarch64`: confirmed as 64-bit buildroot system
- `armv7l`: confirmed as 32-bit buildroot system

2.3 Quick Verification

Select the corresponding executable files and ADLA files for the platform to quickly verify whether the NPU runtime environment is normal.

2.3.1 General Case Verification

The quick verification method in this section is applicable to all Amlogic NPU boards listed in this document, and requires that the system environment of these boards has passed the version verification in Chapter 2.1.

Step 1: Obtain native case

Download link: https://pub-8378326bd0fe4b1d9312a3847f6316a2.r2.dev/Quick_Start/DDK5.10.4.10/ddk_5.10.4.10_native_case.tar.gz

After downloading and extracting, the directory files are as follows

```
├─ 224x224x3_fish_uint8.bin
├─ mobilenet_v1_1.0_224_quant_a311d2.adla
├─ mobilenet_v1_1.0_224_quant_a311y3.adla
├─ mobilenet_v1_1.0_224_quant_c302x2.adla
├─ mobilenet_v1_1.0_224_quant_c305x2.adla
├─ mobilenet_v1_1.0_224_quant_c308l.adla
├─ mobilenet_v1_1.0_224_quant_s905x5.adla
├─ mobilenet_v1_1.0_224_quant_s928x.adla
├─ mobilenet_v1_1.0_224_quant_t968d4.adla
├─ nnsample_android_32
├─ nnsample_android_64
├─ nnsample_buildroot_32
├─ nnsample_buildroot_64
├─ nnsample_yocto_32
└─ nnsample_yocto_64
```

Step 2: Run native case

Android system:

Step 1:

Push the corresponding executable bin file and the corresponding platform adla to the /data/local/tmp directory

Step 2:

#64-bit system

adb shell

cd /data/local/tmp

chmod +x nnsample_android_64

./nnsample_android_64 mobilenet_v1_1.0_224_quant_a311y3.adla

224x224x3_fish_uint8.bin --base

#32-bit system

adb shell

cd /data/local/tmp

chmod +x nnsample_android_32

./nnsample_android_32 mobilenet_v1_1.0_224_quant_a311y3.adla

224x224x3_fish_uint8.bin --base

Buildroot system:

Step 1:

Push the corresponding executable bin file and the corresponding platform adla to the /data/local/tmp directory

Step 2:

```
#64-bit system
adb shell
cd /data/local/tmp
chmod +x nnsample_buildroot_64
./nnsample_buildroot_64 mobilenet_v1_1.0_224_quant_a311y3.adla
224x224x3_fish_uint8.bin --base

#32-bit system
adb shell
cd /data/local/tmp
chmod +x nnsample_android_32
./nnsample_buildroot_32 mobilenet_v1_1.0_224_quant_a311y3.adla
224x224x3_fish_uint8.bin --base
```

Yocto/Armbian system:

Step 1:

Push the corresponding executable bin file and the corresponding platform adla to the /data/local/tmp directory

Step 2:

```
#64-bit system
adb shell
cd /data/local/tmp
chmod +x nnsample_yocto_64
./nnsample_yocto_64 mobilenet_v1_1.0_224_quant_a311y3.adla
224x224x3_fish_uint8.bin --base

#32-bit system
adb shell
cd /data/local/tmp
chmod +x nnsample_yocto_32
./nnsample_yocto_32 mobilenet_v1_1.0_224_quant_a311y3.adla
224x224x3_fish_uint8.bin --base
```

Reference result:

```
2: 0.996094
0: 0.000000
1: 0.000000
3: 0.000000
4: 0.000000
```

Confirm running on NPU:

- **Step 1:** Execute command before and after inference respectively: `cat /proc/interrupts | grep adla`
- **Step 2:** In the two command execution results, the interrupt count in the second column should increase.

```
/data/# cat /proc/interrupts | grep adla
91:          514             0             0             0             0 adla
```

2.3.2 LLM Quick Verification

The quick verification method described in this section is only applicable to the system environments supported by the A311Y3 platform.

Step 1: Obtain LLM Native Case

LLM_Native_case download link: https://pub-8378326bd0fe4b1d9312a3847f6316a2.r2.dev/QuickStart/DDK5.10.4.10/ddk_5.10.4.10_llm_native_case.tar.gz

After downloading and extracting, the directory files are as follows:

```
├─ llm_demo_android_32
├─ llm_demo_android_64
├─ llm_demo_yocto_32
├─ llm_demo_yocto_64
└─ Qwen1.5-0.5B-Chat-Q4AM-PB32.adla
```

Step 2: Run LLM Native Case

Android system:

Step 1:

Push the corresponding executable bin file and the corresponding platform adla to the /data/local/tmp directory

Step 2:

#64-bit system

```
adb shell
```

```
cd /data/local/tmp
```

```
chmod +x llm_demo_android_64
```

```
./llm_demo_android_64 Qwen1.5-0.5B-Chat-Q4AM-PB32.adla --model_type qwen
```

#32-bit system

```
adb shell
```

```
cd /data/local/tmp
```

```
chmod +x llm_demo_android_32
```

```
./llm_demo_android_32 Qwen1.5-0.5B-Chat-Q4AM-PB32.adla --model_type qwen
```

Follow the prompts to perform Q&A

Yocto/Armbian system:

Step 1:

Push the corresponding executable bin file and the corresponding platform adla to the /data/local/tmp directory

Step 2:

#64-bit system

```
adb shell
```

```
cd /data/local/tmp
```

```
chmod +x llm_demo_yocto_64
```

```
./llm_demo_yocto_64 Qwen1.5-0.5B-Chat-Q4AM_PB32.adla --model_type qwen
```

```
#32-bit system
```

```
adb shell
```

```
cd /data/local/tmp
```

```
chmod +x llm_demo_yocto_32
```

```
./llm_demo_yocto_32 Qwen1.5-0.5B-Chat-Q4AM_PB32.adla --model_type qwen
```

Follow the prompts to perform Q&A

3 Toolkit Installation

3.1 Deployment Methods and Toolkit Introduction

This chapter systematically introduces the deployment methods supported by different board-side systems and the supporting toolkits. The following inference methods are currently mainly covered (this document will detail the specific operation flows of Method 1, Method 2, and Method 4)

Method 1: PC-side model conversion + ADB-connected board inference (rapid verification)

- **Process:** Ubuntu PC converts to ADLA via amlnn_toolkit --> remotely invokes board-side NPU through ADB protocol to complete inference
- **Usage:** All series boards covered in this document, requiring Ubuntu host and ADB connection
- **Scenario:** algorithm debugging, quick preview of model effects

Method 2: PC-side model conversion + board-side C/C++ inference (production-grade high-performance deployment)

- **Process:** Ubuntu PC converts to ADLA via amlnn_toolkit --> business code C/C++ cross-compilation --> board-side calls AML Runtime for inference
- **Applicable:** All series boards covered in this document, requiring Ubuntu host and ADB connection (for file transfer)
- **Scenario:** High real-time, Python-independent production environments

Method 3: PC-side model conversion + board-side Lite Python inference (Armbian/Yocto lightweight development)

- **Process:** Ubuntu PC converts to ADLA via amlnn_toolkit --> deploy amlnn_edge_toolkit_lite on board-side --> Python API inference
- **Applicable:** Amlogic NPU boards equipped with Armbian/Yocto systems covered in this document, requiring Ubuntu host and corresponding development board
- **Scenario:** Python rapid development, no cross-compilation required

Method 4: Board-side conversion and inference (edge offline solution without PC dependency)

- **Process:** Install complete amlnn_edge_toolkit on board-side --> local conversion + inference
- **Applicable:** Amlogic NPU boards equipped with Armbian/Yocto systems covered in this document (Yocto requires additional Python and development library dependencies)
- **Scenario:** Offline edge deployment, on-site model iteration

Among the four Amlogic NPU inference deployment solutions introduced above, the three core Python Whl toolkits used are introduced as follows:

amlnn_toolkit

- **Introduction:** PC-side model conversion tool, supporting inference through adb-connected board-side
- **Runtime environment:** Ubuntu PC

amlnn_edge_toolkit

- **Introduction:** Model conversion and inference tool for Armbian/Yocto board-side systems. Limited by the board-side environment, only supports conversion of some models (large

model conversion may cause out-of-memory exceptions), and provides Python interfaces for direct inference.

- **Runtime environment:** Amlogic NPU equipped with Armbian/Yocto systems

amlnn_edge_toolkit_lite

- **Introduction:** ADLA inference tool for Debian/Yocto board-side systems
- **Runtime environment:** Amlogic NPU equipped with Armbian/Yocto systems

3.2 Download AMLNN Related Repositories

For **Method 1**, **Method 2**, and **Method 3** introduced in Section 3.1, model conversion needs to be completed on the PC side, so an Ubuntu 20.04 / 22.04 host needs to be prepared, and the AMLNN related repositories should be downloaded directly in this environment.

Method 4 does not require a PC conversion environment; its conversion and inference are both completed on the board side. Only the required resources need to be downloaded through the GitHub link, and there is no need to pull AMLNN related repositories on the board-side system.

Reference commands for downloading AMLNN related repositories are as follows:

```
# Create project directory

mkdir ~/workspace
cd ~/workspace

# Clone amlnn-toolkit example project repository

git clone https://github.com/Amlogic-NN/amlnn-toolkit.git --depth 1

# Clone AMLNN example project repository

git clone https://github.com/Amlogic-NN/amlnn-model-playground.git --depth 1
```

It is recommended to store the above repositories in the working directory (e.g., ~/workspace/) on the PC side for subsequent model conversion and example running.

3.2.1 amlnn-toolkit Introduction

The overall directory structure is as follows:

```
└─ amlnn-toolkit
  │   └─ amlnn_runtime          # runtime so
  │   └─ amlnn_toolkit
  │       └─ nnserver          # nnserver bin file
  │       └─ whl
  │           └─ aarch64      # amlnn_edge_toolkit whl package
  │           └─ linux_x86   # amlnn_toolkit whl package
  └─ amlnn_toolkit_lite
      └─ whl                  # amlnn_edge_toolkit_lite whl package
  └─ doc                      # AMLNN documentation
  └─ ...
```

3.2.2 amlnn-model-playground Introduction

The overall directory structure is as follows:

```
├─ assets
├─ cmake
├─ common
├─ dependency
├─ docs
├─ examples
│   └─ whisper
│       └─ cpp
│           ├── build-android.sh    # Android system board compilation script
│           ├── build-linux.sh      # Linux system board compilation script
│           └─ src                  # C++ example code
│       ├── input
│       ├── model
│       ├── py                      # python example code
│       └─ yolox
│           ├── cpp
│           ├── build-android.sh    # Android system board compilation script
│           ├── build-linux.sh      # Linux system board compilation script
│           └─ src                  # C++ example code
│       ├── model
│       └─ py                      # python example code
├─ LICENSE
└─ README.md
```

3.3 Install Toolkit Environment

If using **Method 1 / Method 2 / Method 3** (PC-side conversion required): Install amlnn-toolkit on Ubuntu 22.04 LTS host. It is recommended to use Miniforge to create an independent Python 3.10 environment. See Section 3.3.1 for details.

If using **Method 4** (full board-side workflow): Install amlnn_edge_toolkit directly on the Armbian board side. It is also recommended to use Miniforge to create an independent Python 3.10 environment. See Section 3.3.2 for details.

3.3.1 Ubuntu PC-side amlnn-toolkit Installation

For inference scenarios requiring model conversion on the PC side, follow the steps below to install amlnn-toolkit on the Ubuntu system.

Step 1: Install Miniforge

```
wget https://github.com/conda-
forge/miniforge/releases/latest/download/Miniforge3-Linux-x86_64.sh

bash Miniforge3-Linux-x86_64.sh

# Follow the prompts, then reload configuration
source ~/.bashrc
```

Step 2: Create and activate Python virtual environment

```
conda create -n amlnn python=3.10 -y
conda activate amlnn

python --version
pip install --upgrade pip
```

Step 3: Install amlnn_toolkit

The whl package is located in the `~/workspace/amlnn-toolkit/amlnn_toolkit/whl/linux_x86/` directory.

```
# Ensure amlnn environment is activated
conda activate amlnn

# Enter amlnn-toolkit directory
cd ~/workspace/amlnn-toolkit

# Install AMLNN-Toolkit dependencies and package
for req in $(cat amlnn_toolkit/whl/linux_x86/requirements_x86_64.txt); do pip
install $req; done
pip install amlnn_toolkit/whl/linux_x86/amlnn_toolkit-1.0.0-cp310-cp310-
linux_x86_64.whl
```

Step 4: Verify installation

```
python -c "from amlnn.api import AMLNN; print('AMLNN-Toolkit import OK')"
```

Successfully prints AMLNN-Toolkit import OK

3.3.2 Board-side Armbian System Installation of amlnn-edge-toolkit

For scenarios requiring direct model conversion and inference on the board side, follow the steps below to install `amlnn-edge-toolkit` on the Armbian system. To save board-side storage space, please directly download the precompiled files from the `amlnn_toolkit/whl/aarch64/` directory of the [AMLNN Toolkit](#) repository, including:

- `requirements_aarch64.txt`
- `amlnn_edge_toolkit-1.0.0-cp310-cp310-linux_aarch64.whl`

After downloading, execute the following commands to install:

Step 1: Install Miniforge

```
wget https://github.com/conda-
forge/miniforge/releases/latest/download/Miniforge3-Linux-aarch64.sh

bash Miniforge3-Linux-aarch64.sh

# Follow the prompts, then reload configuration
source ~/.bashrc
```

Step 2: Create and activate Python virtual environment

```
conda create -n amlnn python=3.10 -y
conda activate amlnn

python --version
pip install --upgrade pip
```

Step 3: Install amlnn_edge_toolkit

```
# Ensure amlnn environment is activated
conda activate amlnn

# Install AMLNN-Toolkit dependencies and package
for req in $(cat requirements_aarch64.txt); do pip install $req; done
pip install amlnn_edge_toolkit-1.0.0-cp310-cp310-linux_aarch64.whl
```

Step 4: Verify installation

```
python -c "from amlnn.api import AMLNN; print('AMLNN-Toolkit import OK')"
```

Successfully prints AMLNN-Toolkit import OK

4 Board-side Conversion and Inference

This section is applicable to the Amlogic NPU board environments equipped with Armbian/Yocto systems covered in this document. Before starting board-side conversion and inference using the YOLOX object detection model as a practical case, please be sure to refer to "Section 3.3.2: Installing amlnn_edge_toolkit on Board-side Armbian" to complete environment deployment, and then follow the current section process to achieve board-side model conversion and inference full workflow.

4.1 Preparation

To save board-side storage space, it is recommended to directly download the required resources from this link: <https://github.com/Amlogic-NN/amlnn-model-playground/tree/main/examples/yolox>

Step 1: Download model

```
wget https://pub-8378326bd0fe4b1d9312a3847f6316a2.r2.dev/model_zoo/yolox/yolox_s.onnx
```

Step 2: Prepare quantization dataset

Due to license restrictions, please refer to <https://github.com/Amlogic-NN/amlnn-model-playground/tree/main/resource> to prepare the detection_dataset quantization dataset.

Step 3: Download conversion script and test files

- Visit [amlnn-model-playground/examples/yolox/py](https://github.com/Amlogic-NN/amlnn-model-playground/tree/main/examples/yolox/py), to obtain export_adla.py and yolox_inference.py
- Visit [amlnn-model-playground/examples/yolox/input](https://github.com/Amlogic-NN/amlnn-model-playground/tree/main/examples/yolox/input), download test.png, and save it to the local input directory

4.2 Model Conversion and Inference

In the board-side environment, follow the steps below to complete YOLOX model ADLA conversion and inference verification.

Step 1: Model conversion (ONNX to ADLA)

Execute the export_adla.py script to convert the ONNX model to the ADLA format adapted to the target platform.

```
python export_adla.py \  
    --onnx yolox_s.onnx \  
    --dataset-path ../../../../resource/detection_dataset.txt \  
    --target-platform 007 \  
    --adla .
```

Step 2: Model inference

After conversion, use the generated ADLA model for inference testing.

```
python yolox_inference.py \  
    --model-path yolox_s_w8a8.adla \  
    --image-dir input
```

Script description: In the example project of the current amlnn-model-playground repository, export_adla.py (model conversion script) and yolox_inference.py (model inference script) are two independent files, both developed based on board-side AMLNN native interfaces. They can be integrated into a unified script to achieve one-click ONNX-to-ADLA conversion + inference verification full workflow. For specific integration implementation, please refer to the example code in Section 4.3 of this document.

4.3 USB Camera Real-time Acquisition Demo Acquisition Guide

This Demo implements full-process ADLA deployment of the YOLOX model for Amlogic development boards, enabling USB camera real-time streaming and edge-side object detection. The acquisition method is as follows:

Step 1: Download resource package and extract

```
wget https://pub-  
8378326bd0fe4b1d9312a3847f6316a2.r2.dev/Quick_Start/examples/yolox_camera.tar.gz  
tar -zxvf yolox_camera.tar.gz
```

Step 2: Deployment instructions

After extraction, directly refer to README_CN.md (Chinese guide) or README.md (English guide) in the directory to complete the full-process configuration and running.

5 PC-side Model Conversion

This chapter uses the YOLOX object detection model as a practical case to systematically explain how to use the amlnn-toolkit toolchain to convert open-source models into ADLA format board-side inference files. On this basis, combined with Chapter 6 and Chapter 7, it introduces two inference methods described in Section 3.1:

- **Method 1:** After PC-side model conversion, perform board-connected inference through ADB;
- **Method 2:** After PC-side model conversion, run C/C++ inference programs on the board side.

It should be specifically noted that before PC-side model conversion, please be sure to refer to "**Section 3.3.1: Installing amlnn-toolkit on Ubuntu PC**" to complete environment deployment, and then follow the current section process to perform conversion operations.

5.1 Preparation

5.1.1 Download Model

```
cd ~/workspace/amlnn-model-playground/examples/yolox/model

wget https://pub-8378326bd0fe4b1d9312a3847f6316a2.r2.dev/model_zoo/yolox/yolox_s.onnx
```

The `README.md` files in each subdirectory under the `examples` directory, except for a few models whose acquisition methods are not provided due to license restrictions, all detail the ONNX model acquisition methods or export steps.

5.1.2 Platform Type Confirmation

Before starting the ADLA model conversion process, the board model must first be confirmed to obtain the `target_platform` parameter that fully matches the hardware, avoiding conversion failures or inference abnormalities due to platform incompatibility. Currently, the final `target_platform` value can be queried through the following two independent methods, and all verification results can be checked against the "**Part_number/Serial/target_platform Correspondence Table**" below. The correspondence table is as follows:

Part_number	Serial	target_platform
C308L / C302X	Serial: 3d0a 010100000000c61349	PRODUCT_PID0XA001
S928X	Serial: 3e0a 0202000000007c404b	PRODUCT_PID0XA002
A311D2	Serial: 360c 01030000000025f904	PRODUCT_PID0XA003
T968D4	Serial: 420a 010100000000daff6e	PRODUCT_PID0XA004
S905X5 / S905D5	Serial: 480a 020100000000682763	PRODUCT_PID0XA005
C302X2	Serial: 4c0a 01030000000055665a	PRODUCT_PID0XA006
A311Y3	Serial: 4d0a 01020000000070e84a	PRODUCT_PID0XA007

Part_number	Serial	target_platform
C305X2/A123X	Serial: 4f0a 01010000000054c6d1	PRODUCT_PID0XA008

Method 1: Quick confirmation via Part_number

Directly read the Part_number identifier engraved on the board hardware surface, match the read number with the correspondence table, and obtain the correct target_platform parameter corresponding to the board.

Method 2: Confirmation via Serial number verification

Obtain Serial information: After the board starts normally, connect to the device via ADB and execute the following commands to read the Serial number:

```
adb shell cat /proc/cpuinfo
# or
adb shell cat /proc/cpu_chipid
```

Matching rules:

- **Preferred rule:** Priority is to intercept **the first 4 characters** of the Serial number, compare with the correspondence table to complete matching, and obtain the target `target_platform` value.
- **Alternative rule:** If the first 4 characters have no matching result, you can downgrade to intercepting **the first 3 characters** of the Serial number to complete matching, and finally obtain the correct `target_platform` parameter.

5.2 Model Conversion

Model conversion

```
cd ~/workspace/amlnn-model-playground/examples/yolox/py

python export_adla.py --onnx ../model/yolox_s.onnx \
    --dataset-path ../../resource/detection_dataset.txt \
    --target-platform 007 \
    --adla ../model
```

Parameter description

Parameter	Description
<code>--onnx</code>	<code>.onnx</code> model path
<code>--dataset-path</code>	Quantization calibration dataset
<code>--target-platform</code>	Specifies the board type parameter, whose value needs to be consistent with the last three digits of the target_platform value confirmed in Section 5.1.2 .
<code>--adla</code>	Specifies the output path of the adla file, default value: ../model

- The current conversion script `export_adla.py` has built-in automatic adaptation logic for multiple parameters. If you need to confirm the actual calling parameters of AMLNN, please directly refer to the AMLNN-related operation implementation part in the script.
- `detection_dataset.txt`: Due to license restrictions, please refer to <https://github.com/Amlogic-NN/amlnn-model-playground/tree/main/resource> to prepare the `detection_dataset` quantization dataset.

Conversion result: The `yolox_s_w8a8.adla` model file exists in the `examples/yolox/model` directory

6 ADB Board-connected Inference

Before performing ADB board-connected inference, please be sure to refer to "**Chapter 5: PC-side Model Conversion**" to complete the conversion of the ADLA model file, and then follow the current chapter process to complete ADB board-connected inference.

ADB board-connected inference refers to a remote inference solution where the Ubuntu PC side establishes a communication link with the development board through the ADB protocol, transfers inference data, and finally completes computation on the board-side NPU. This solution relies on the board-side nn_server background proxy service being started in advance. The core component definitions are as follows:

- **nnserver**: Development board-side background proxy service, responsible for communication between the PC-side Demo and the board-side NPU. The PC-side Python program can send inference requests and input data via USB/network. After receiving, nnserver calls the ADLA Runtime to complete model loading, NPU inference, and returns results. Before using the PC-side Demo, model verification, or board-connected debugging, this service must be started on the development board side first.
- **AML Runtime (libnnsdk.so)**: The underlying runtime component, responsible for loading and managing models (computational graphs), scheduling the NPU to complete inference computation, and providing core capabilities such as I/O buffer management, compilation/execution scheduling, etc., providing underlying support for upper-layer services such as nnserver.

6.1 Install ADB Tool

To use the ADB board-connected inference method, ensure that ADB is installed on the Ubuntu PC side. ADB (Android Debug Bridge) is a commonly used tool for communication and debugging between the PC side and development boards. Ubuntu PC ADB installation guide is as follows:

```
sudo apt-get update
sudo apt-get install adb

# Verify installation

adb version

# After connecting the development board, check whether the device is recognized
adb devices

# Expected output:
#   List of devices attached
#       1234567890   device
```

6.2 Start nnserver

Step 1: Obtain nnserver file

```
cd ~/workspace/amlnn-toolkit/
```

Based on the system environment confirmed in [2.2 System Environment and Architecture Verification], select the corresponding nnserver file.

- **Android 64-bit:** amlnn_toolkit/nnserver/android/lib64/nnserver
- **Android 32-bit:** amlnn_toolkit/nnserver/android/lib32/nnserver
- **Armbian/Yocto 64-bit:** amlnn_toolkit/nnserver/linux/yocto/lib64/nnserver
- **Armbian/Yocto 32-bit:** amlnn_toolkit/nnserver/linux/yocto/lib32/nnserver
- **Buildroot 64-bit:** amlnn_toolkit/nnserver/linux/buildroot/lib64/nnserver
- **Buildroot 32-bit:** amlnn_toolkit/nnserver/linux/buildroot/lib32/nnserver

Step 2: Start nnserver

```
adb shell "mkdir -p /data/nn"
adb push /path/to/nnserver /data/nn/
adb shell "chmod +x /data/nn/nnserver"
adb shell "cd /data/nn && ./nnserver"

# If information similar to the following is displayed, the nnserver service has
started successfully
NNSERVER, vx.x.x, 20xx.xx
Coordinator listening on port 8307
Server listening on port 8308
Server listening on port 8309
server listening on port 8310
```

6.3 Run AMLNN Python Demo

Enter the example directory and edit yolox_inference.py to switch the runtime mode from local native mode to NNServer mode, and enable performance statistics to monitor inference latency. The operation steps are as follows:

Step 1: Enter directory and open target file

```
cd ~/workspace/amlnn-model-playground/examples/yolox/py
vi yolox_inference.py # locate the `amlnn.init_runtime()` call
```

Step 2: Modify code

```
# Before modification
# amlnn.init_runtime(mode="native", enable_perf=True)

# After modification
amlnn.init_runtime(mode="nnserver", enable_perf=True)
```

Step 3: Execute inference test

```
python yolox_inference.py --model-path ../model/yolox_s_w8a8.adla --image-dir
../input
```

Step 4: Confirm results

The inference result will be automatically saved as a test_result.jpg file. You can pull this file to the local side via ADB to view it. For specific detection results, please refer to the image in **Chapter 7.2.4** of this document.

7 Board-side C/C++ Inference

Before performing board-side C/C++ inference, please be sure to first refer to "**Chapter 5: PC-side Model Conversion**" to complete the conversion of the ADLA model file, and then follow the workflow in this chapter to complete cross-compilation of the C/C++ Demo on the PC side and push the generated executable file to the board side for execution. The standard AMLNN C/C++ Demo end-to-end deployment usually covers the following three core steps:

- **Source code compilation:** Use the cross-compilation toolchain on the PC side to compile the Demo source code, generating executable files adapted to the target board-side architecture;
- **Resource deployment:** Transfer the generated executable files, converted model files, and test images and other dependency resources to the development board's specified directory via ADB or network;
- **Board-side execution:** Run the Demo program in the development board terminal environment, verifying inference output results and performance metrics.

7.1 Install Compilation Toolchain

7.1.1 Install CMake

CMake is a cross-platform build system generator tool used to generate Makefiles or project files. The minimum version requirement is CMake 3.10+, and CMake 3.20+ is recommended.

```
sudo apt-get update
sudo apt-get install cmake

# Verify version

cmake --version

# Output: cmake version 3.x.x
# Minimum version requirement is CMake 3.10+, recommended CMake 3.20+

# If a higher version is needed, install manually

wget https://github.com/Kitware/CMake/releases/download/v3.27.0/cmake-3.27.0-
linux-x86_64.sh

sudo bash cmake-3.27.0-linux-x86_64.sh --prefix=/usr/local --skip-license

cmake --version
```

7.1.2 Install Cross-compilation Toolchain

Install the matching cross-compilation toolchain according to the operating system type determined in "**Chapter 2.2 System Environment and Architecture Verification**".

The cross-compilation toolchain is the core bridge connecting the development host (Host) and the target board (Target), responsible for generating executable files on the high-performance PC side that adapt to the board-side CPU instruction set and ABI specifications. Please select the corresponding installation scheme below based on the system environment confirmed in

previous chapters.

7.1.2.1 Android System: Install and Configure NDK

When the development board is an Android system, the NDK toolchain needs to be installed and configured.

Step 1: Android NDK download link: <https://github.com/android/ndk/wiki/Unsupported-Downloads>

```
cd ~/workspace

# ===== Download and configure NDK =====
wget https://dl.google.com/android/repository/android-ndk-r25c-linux.zip
unzip android-ndk-r25c-linux.zip

echo 'export ANDROID_NDK_PATH=$HOME/workspace/android-ndk-r25c' >> ~/.bashrc
echo 'export PATH=$ANDROID_NDK_PATH:$PATH' >> ~/.bashrc
source ~/.bashrc
```

Step 2: Verify installation:

```
ndk-build --version
#If version information can be output normally, it indicates that NDK is
installed successfully
```

7.1.2.2 Buildroot System: Install GCC Cross-compilation Toolchain

When the development board is a Buildroot system, install the GCC cross-compilation toolchain matching the board-side Release.

Note:

- Buildroot is relatively sensitive to toolchain version, glibc, libstdc++, and ABI compatibility. It is recommended to prioritize using the cross-compilation toolchain included with or explicitly specified by the corresponding board-side Release.
- The following toolchains are reference versions. If the project Release Notes specify otherwise, please follow the Release Notes.
- Before starting installation, please first confirm whether the board is a 32-bit or 64-bit system according to "Chapter 2.2 System Environment and Architecture Verification".

For 64-bit System:

```
cd ~/workspace

wget https://developer.arm.com/-/media/Files/downloads/gnu-a/10.3-2021.07/binrel/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.xz

tar -xf gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.xz

# Configure toolchain environment variables (only valid for current terminal)
```

```
export BUILDROOT_TOOLCHAIN=$HOME/workspace/gcc-arm-10.3-2021.07-x86_64-aarch64-  
none-linux-gnu  
export PATH=$BUILDROOT_TOOLCHAIN/bin:$PATH  
  
# verify installation  
aarch64-none-linux-gnu-gcc --version  
aarch64-none-linux-gnu-g++ --version
```

For 32-bit System:

```
cd ~/workspace  
  
wget https://armkeil.blob.core.windows.net/developer/Files/downloads/gnu-a/10.3-  
2021.07/binrel/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi.tar.xz  
  
tar -xf gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi.tar.xz  
  
# Configure toolchain environment variables (only valid for current terminal)  
export BUILDROOT_TOOLCHAIN=$HOME/workspace/gcc-arm-10.3-2021.07-x86_64-arm-none-  
linux-gnueabi  
export PATH=$BUILDROOT_TOOLCHAIN/bin:$PATH  
  
# verify installation  
arm-none-linux-gnueabi-gcc --version  
arm-none-linux-gnueabi-g++ --version  
  
If you want the environment variables to take effect permanently, you can write  
the corresponding export commands to ~/.bashrc, then execute:  
  
source ~/.bashrc
```

7.1.2.3 Armbian/Yocto System: Install Cross-compilation Toolchain

When the development board is an Armbian/Yocto system, the corresponding cross-compilation toolchain needs to be installed and configured.

For 64-bit System:

```
wget https://pub-  
8378326bd0fe4b1d9312a3847f6316a2.r2.dev/toolchain/yocto_toolchain/64/poky-glibc-  
x86_64-meta-toolchain-armv8a-meson7-an400-5.15-a64-toolchain-4.0.20.sh  
  
./poky-glibc-x86_64-meta-toolchain-armv8a-meson7-an400-5.15-a64-toolchain-  
4.0.20.sh
```

For 32-bit System:

```
wget https://pub-  
8378326bd0fe4b1d9312a3847f6316a2.r2.dev/toolchain/yocto_toolchain/32/poky-glibc-  
x86_64-amlogic-bsp-armv7at2hf-neon-meson7-bh201-5.15-a32-toolchain-4.0.20.sh  
  
./poky-glibc-x86_64-amlogic-bsp-armv7at2hf-neon-meson7-bh201-5.15-a32-toolchain-  
4.0.20.sh
```

7.2 Run AMLNN C/C++ Demo

7.2.1 Android System C/C++ Demo Compilation and Execution

Step 1: Compile Android executable file

For 64-bit System:

```
cd ~/workspace/aml-nn-model-playground/examples/yolox/cpp/  
  
# Default compile 64-bit arm64-v8a  
  
./build-android.sh  
  
ls build/android/  
  
# yolox_demo
```

For 32-bit System:

```
cd ~/workspace/aml-nn-model-playground/examples/yolox/cpp/  
  
./build-android.sh -a armeabi-v7a  
  
ls build/android/  
#yolox_demo
```

Step 2: Push and run

```
cd ~/workspace/aml-nn-model-playground/examples/yolox/cpp/  
  
adb shell "mkdir -p /data/local/tmp/yolox"  
  
adb push build/android/yolox_demo /data/local/tmp/yolox/  
adb push ../model/yolox_s_w8a8.adla /data/local/tmp/yolox/  
adb push ../input /data/local/tmp/yolox/
```

Run on board:

```
adb shell  
cd /data/local/tmp/yolox  
chmod +x yolox_demo  
  
# 64-bit Android  
export LD_LIBRARY_PATH=/vendor/lib64:$LD_LIBRARY_PATH  
  
# For 32-bit Android, change to:  
# export LD_LIBRARY_PATH=/vendor/lib:$LD_LIBRARY_PATH  
  
# Usage: ./yolox_demo <model_path> <image_dir>  
./yolox_demo yolox_s_w8a8.adla input
```

7.2.2 Buildroot System C/C++ Demo Compilation and Execution

Step 1: Compile Buildroot executable file

For 64-bit System:

```
export AMLNN_HOME=$HOME/workspace/amlnn-toolkit/amlnn_runtime
export BUILDROOT_TOOLCHAIN=$HOME/workspace/gcc-arm-10.3-2021.07-x86_64-aarch64-
none-linux-gnu
export PATH=$BUILDROOT_TOOLCHAIN/bin:$PATH

cd ~/workspace/amlnn-model-playground/examples/yolox/cpp/

./build-linux.sh

# Default generates 64-bit program

ls build/linux/64/yolox_demo
```

For 32-bit System:

```
export AMLNN_HOME=$HOME/workspace/amlnn-toolkit/amlnn_runtime
export BUILDROOT_TOOLCHAIN=$HOME/workspace/gcc-arm-10.3-2021.07-x86_64-arm-none-
linux-gnueabi
export PATH=$BUILDROOT_TOOLCHAIN/bin:$PATH

cd ~/workspace/amlnn-model-playground/examples/yolox/cpp/

./build-linux.sh -b 32

ls build/linux/32/yolox_demo
```

Step 2: Push files

The following uses /data/local/tmp/yolox as an example directory. If the current Buildroot system does not have /data/local/tmp, you can replace it with /root/yolox or other writable directories.

64-bit system:

```
cd ~/workspace/amlnn-model-playground/examples/yolox/cpp/

adb shell "mkdir -p /data/local/tmp/yolox"

adb push build/linux/64/yolox_demo /data/local/tmp/yolox/
adb push ../model/yolox_s_w8a8.adla /data/local/tmp/yolox/
adb push ../input /data/local/tmp/yolox/
```

For 32-bit systems, just replace the executable file path with:

```
adb push build/linux/32/yolox_demo /data/local/tmp/yolox/
```

Step 3: Run on board

```
adb shell
cd /data/local/tmp/yolox
chmod +x yolox_demo

# Check executable file and dynamic library dependencies
file yolox_demo
ldd yolox_demo
```

If ldd shows libnnsdk.so => not found, please set according to the actual library directory on the board:

```
export LD_LIBRARY_PATH=/usr/lib:/usr/lib64:$LD_LIBRARY_PATH
```

Run Demo:

```
./yolox_demo yolox_s_w8a8.adla input
```

7.2.3 Armbian/Yocto System C/C++ Demo Compilation and Execution

Both Armbian and Yocto systems use build-linux.sh to compile Linux version C/C++ Demo. Before starting compilation, please complete the installation and environment configuration of the corresponding cross-compilation toolchain.

Step 1: Configure AMLNN Runtime path

```
export AMLNN_HOME=$HOME/workspace/amlnn-toolkit/amlnn_runtime
cd ~/workspace/amlnn-model-playground/examples/yolox/cpp
```

Step 2: Compile executable file

Armbian 64-bit system:

```
./build-linux.sh
```

Armbian 32-bit system:

```
./build-linux.sh -b 32
```

Yocto 64-bit system:

```
./build-linux.sh -m yocto -s /path/to/yocto_sdk_root -t /path/to/toolchain.cmake
```

Yocto 32-bit system:

```
./build-linux.sh -m yocto -b 32 -s /path/to/yocto_sdk_root -t
/path/to/toolchain.cmake
```

Compilation outputs are generated in the following directories by default:

```
# Armbian
build/linux/64/yolox_demo
build/linux/32/yolox_demo

# Yocto
build/yocto/64/yolox_demo
build/yocto/32/yolox_demo
```

Step 3: Push files to board

The following uses 64-bit system as an example:

```
adb shell "mkdir -p /data/local/tmp/yolox"

# Armbian
adb push build/linux/64/yolox_demo /data/local/tmp/yolox/

# Yocto
# adb push build/yocto/64/yolox_demo /data/local/tmp/yolox/

adb push ../model/yolox_s_w8a8.adla /data/local/tmp/yolox/
adb push ../input /data/local/tmp/yolox/
```

Step 4: Run on board

```
adb shell
cd /data/local/tmp/yolox
chmod +x yolox_demo

# Usage: ./yolox_demo <model_path> <image_dir>
./yolox_demo yolox_s_w8a8.adla input
```

7.2.4 Expected Results

By default, the detection result image will be saved to:

```
model_result/<test_image_name>_result.jpg
```

You can pull the detection result to the PC side via ADB to view:

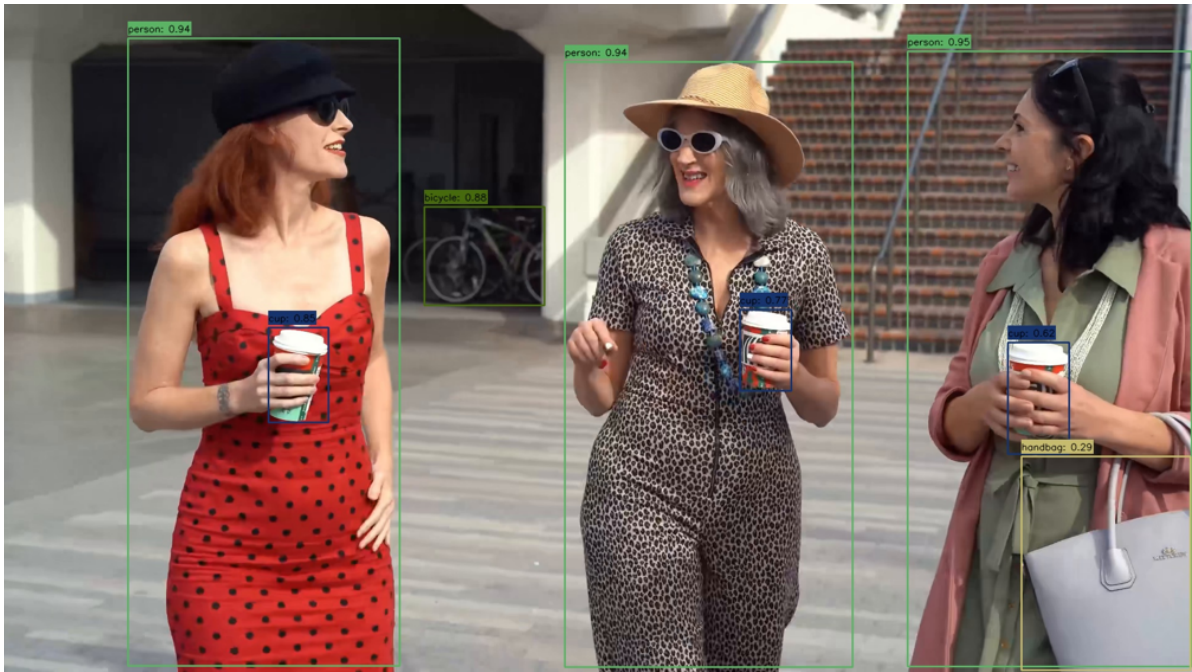
```
adb pull /data/local/tmp/yolox/model_result ./model_result
```

If Armbian uses /root/yolox as the running directory, execute:

```
adb pull /root/yolox/model_result ./model_result
```

Or use SCP:

```
scp -r root@<board-ip>:/root/yolox/model_result ./
```



8 FAQ

8.1 adb devices Cannot Recognize Device

You can try the following methods to troubleshoot:

1. Check whether the data cable connection is correct, try replugging the data cable, changing the computer USB port, or replacing the data cable.
2. When connecting the development board via USB, ensure that only one adb server service is running on both the local computer and the Docker container at the same time. For example, if you need to connect the development board in a Docker container, please first execute `adb kill-server` in the computer terminal to terminate the local adb server.
3. If it prompts command 'adb' not found, it means adb is not installed in the system, and you need to execute `sudo apt install adb` to install it.

8.2 nnserver Service Related Issues

For the function description, deployment method, startup parameters, port configuration, and common problem troubleshooting of nnserver, please refer to the supporting document **"10.NNServer.md"**. It is recommended to consult this first during board-connected debugging or service connection abnormalities.